

```
// server.js

// IMPORTAR LIBRERIA
const express = require('express');

const app = express();
const PORT = 3000;

// MIDDLEWARE PARA LEER JSON
app.use(express.json());
```

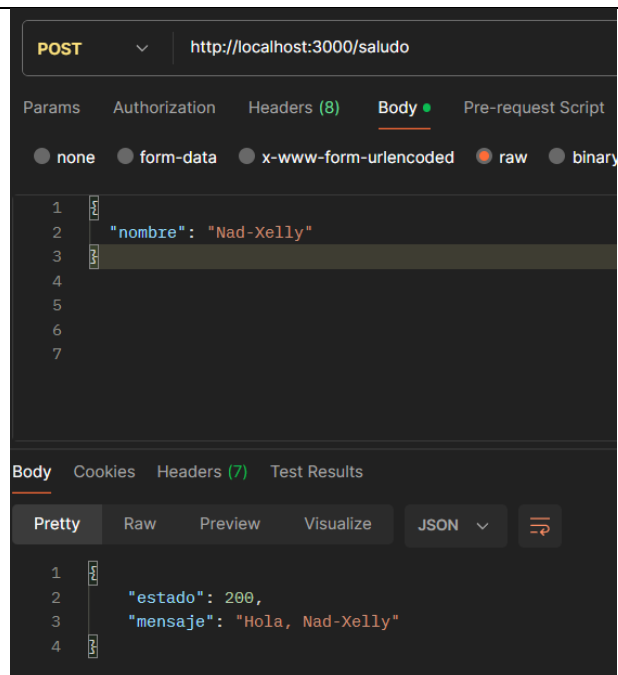
Ejercicio 1: Servicio de Saludo Básico

// Crear un endpoint POST /saludo que reciba { "nombre": "string" } // y responda { "estado": valor, "mensaje": "Hola, [nombre]" }

```
/* =====
EJERCICIO 1: SALUDO
===== */
app.post('/saludo', (req, res) => {
  try {
    const { nombre } = req.body;

    if (!nombre || typeof nombre !== 'string') {

      return res.status(400).json({
        estado: 400,
        mensaje: "Nombre no proporcionado o
inválido"
      });
    }
    res.status(200).json({
      estado: 200,
      mensaje: `Hola, ${nombre}`
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      mensaje: "Error interno del servidor"
    });
  }
});
```

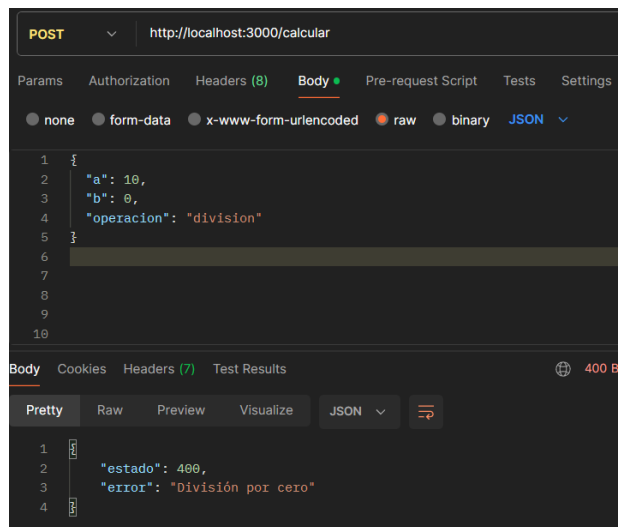
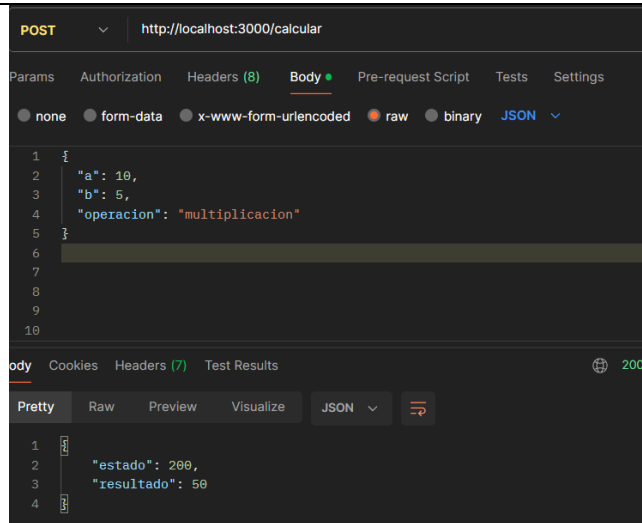


Ejercicio 2: Calculadora de Operaciones Básicas

// Endpoint POST /calcular que reciba { "a": number, "b": number, "operacion": "suma|resta|multiplicacion|division" } // Responder { "estado": valor, "resultado": number } o { "error": "mensaje" } en caso de división por cero

```
/* =====
EJERCICIO 2: CALCULADORA
===== */
app.post('/calcular', (req, res) => {
  try {
    const { a, b, operacion } = req.body;
    if (typeof a !== 'number' || typeof b !==
'number') {
      return res.status(400).json({
        estado: 400,
        error: "a y b deben ser números"
      });
    }
    let resultado;
    switch (operacion) {
      case 'suma':
        resultado = a + b;
        break;
      case 'resta':
        resultado = a - b;
        break;
      case 'multiplicacion':
        resultado = a * b;
        break;
      case 'division':

        if (b === 0) {
          return res.status(400).json({
            estado: 400,
            error: "División por cero"
          });
        }
        resultado = a / b;
        break;
      default:
        return res.status(400).json({
          estado: 400,
          error: "Operación no válida"
        });
    }
    res.status(200).json({
      estado: 200,
      resultado
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      error: "Error en el cálculo"
    });
  }
});
```



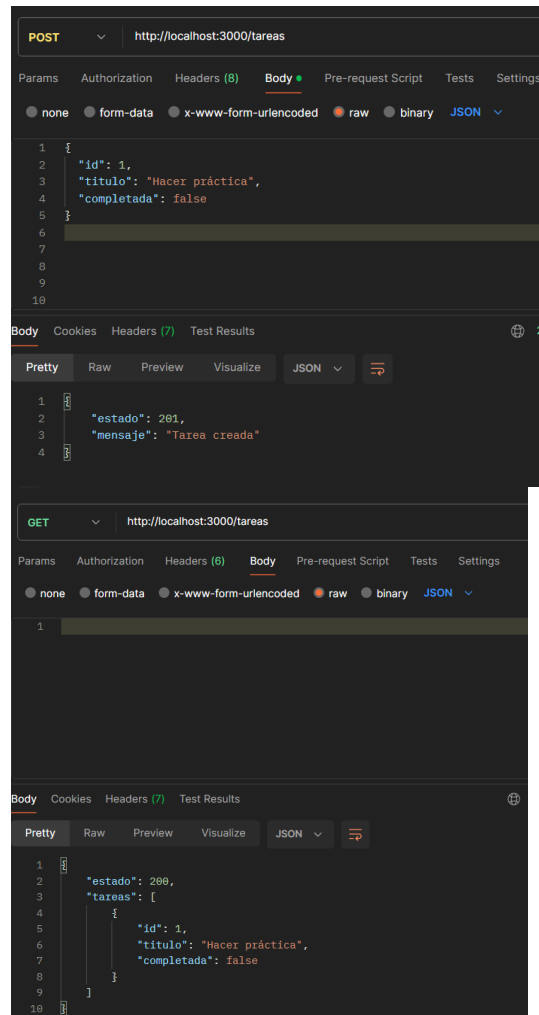
Ejercicio 3: Gestor de Tareas (CRUD Básico)

// Implementar un CRUD simple para tareas: // - POST /tareas: Crear tarea { "id": number, "titulo": string, "completada": boolean } // - GET /tareas: Listar todas las tareas // - PUT /tareas/:id: Actualizar una tarea // - DELETE /tareas/:id: Eliminar una tarea // Usar un array en memoria para almacenar las tareas Agregar un campo en la respuesta: "estado": valor

```

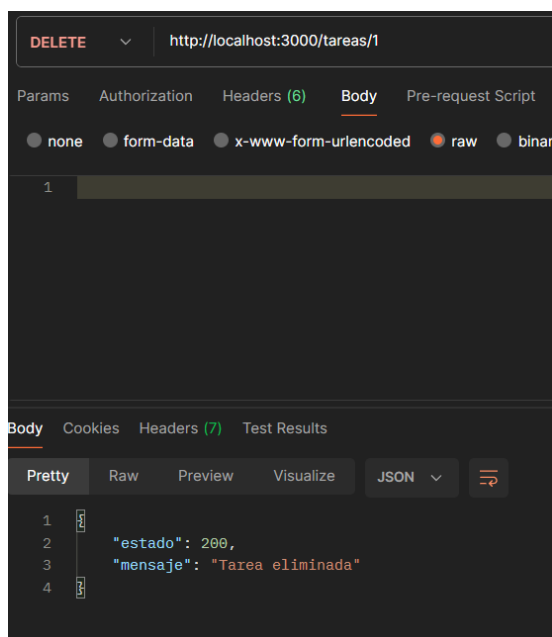
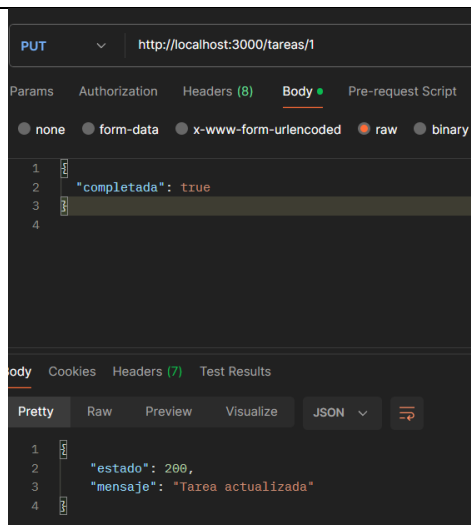
/* =====
EJERCICIO 3: CRUD TAREAS
===== */
let tareas = [];
app.post('/tareas', (req, res) => {
  try {
    const { id, titulo, completada } = req.body;
    if (typeof id !== 'number' || typeof titulo !== 'string' ||
    typeof completada !== 'boolean') {
      return res.status(400).json({
        estado: 400,
        mensaje: "Datos inválidos"
      });
    }
    tareas.push({ id, titulo, completada });
    res.status(201).json({
      estado: 201,
      mensaje: "Tarea creada"
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      mensaje: "Error al crear tarea"
    });
  }
});
app.get('/tareas', (req, res) => {
  res.status(200).json({
    estado: 200,
    tareas
  });
});
app.put('/tareas/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const index = tareas.findIndex(t => t.id === id);
    if (index === -1) {
      return res.status(404).json({
        estado: 404,
        mensaje: "Tarea no encontrada"
      });
    }
    tareas[index] = {
      ...tareas[index],
      ...req.body
    };
    res.status(200).json({
      estado: 200,
      mensaje: "Tarea actualizada"
    });
  }
});

```



Práctica de repaso para API Restful

```
} catch (error) {
  res.status(500).json({
    estado: 500,
    mensaje: "Error al actualizar tarea"
  });
}
});
app.delete('/tareas/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const existe = tareas.some(t => t.id === id);
    if (!existe) {
      return res.status(404).json({
        estado: 404,
        mensaje: "Tarea no encontrada"
      });
    }
    tareas = tareas.filter(t => t.id !== id);
    res.status(200).json({
      estado: 200,
      mensaje: "Tarea eliminada"
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      mensaje: "Error al eliminar tarea"
    });
  }
});
```



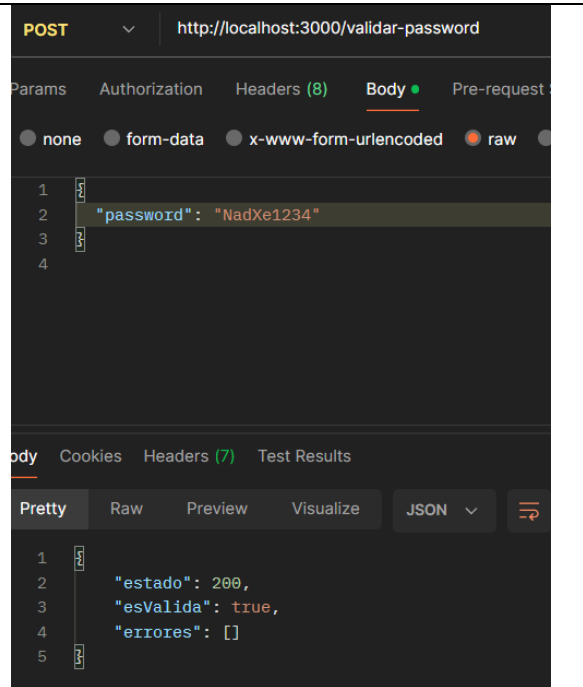
Ejercicio 4: Validador de Contraseñas

// Endpoint POST /validar-password que reciba { "password": "string" } // Validar: mínimo 8 caracteres, al menos una mayúscula, una minúscula y un número // Responder { "estado": valor, "esValida": boolean, "errores": string[] }

```
/* =====
EJERCICIO 4: VALIDAR PASSWORD
===== */

app.post('/validar-password', (req, res) => {
  try {
    const { password } = req.body;
    if (!password || typeof password !== 'string') {
      return res.status(400).json({
        estado: 400,
        mensaje: "Password inválido"
      });
    }
    const errores = [];
    if (password.length < 8)
      errores.push("Mínimo 8 caracteres");
    if (!/[A-Z]/.test(password))
      errores.push("Al menos una mayúscula");

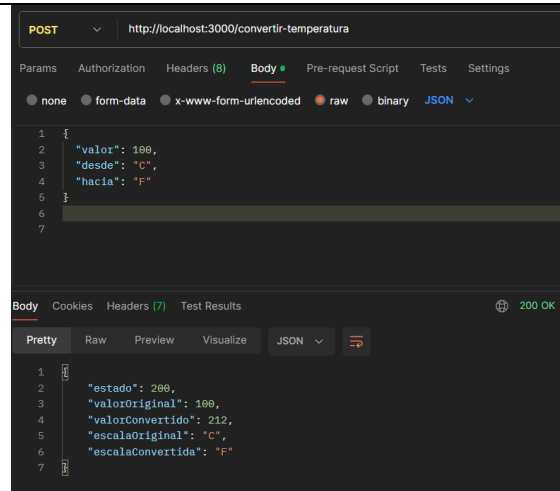
    if (!/[a-z]/.test(password))
      errores.push("Al menos una minúscula");
    if (!/\d/.test(password))
      errores.push("Al menos un número");
    res.status(200).json({
      estado: 200,
      esValida: errores.length === 0,
      errores
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      mensaje: "Error interno"
    });
  }
});
```



Ejercicio 5: Conversor de Temperatura

// Endpoint POST /convertir-temperatura que reciba { "valor": number, "desde": "C|F|K", "hacia": "C|F|K" } // Responder { "estado": number, "valorOriginal": number, "valorConvertido": number, "escalaOriginal": string, "escalaConvertida": string }

```
/* =====  
EJERCICIO 5: TEMPERATURA  
===== */  
  
app.post('/convertir-temperatura', (req, res) => {  
  
  try {  
  
    let { valor, desde, hacia } = req.body;  
  
    if (typeof valor !== 'number') {  
  
      return res.status(400).json({  
        estado: 400,  
        mensaje: "Valor inválido"  
      });  
    }  
    let tempC;  
    if (desde === 'C') tempC = valor;  
    else if (desde === 'F') tempC = (valor - 32) * 5/9;  
    else if (desde === 'K') tempC = valor - 273.15;  
    else {  
      return res.status(400).json({  
        estado: 400,  
        mensaje: "Escala inválida"  
      });  
    }  
    let resultado;  
    if (hacia === 'C') resultado = tempC;  
    else if (hacia === 'F') resultado = (tempC * 9/5) + 32;  
    else if (hacia === 'K') resultado = tempC + 273.15;  
    else {  
      return res.status(400).json({  
        estado: 400,  
        mensaje: "Escala inválida"  
      });  
    }  
    res.status(200).json({  
      estado: 200,  
      valorOriginal: valor,  
      valorConvertido: resultado,  
      escalaOriginal: desde,  
      escalaConvertida: hacia  
    });  
  } catch (error) {  
    res.status(500).json({  
      estado: 500,  
      mensaje: "Error interno"  
    });  
  }  
});
```



Ejercicio 6: Buscador en Array

// Endpoint POST /buscar que reciba { "array": any[], "elemento": any } // Responder { "estado": valor, "encontrado": boolean, "indice": number, "tipoElemento": string }

```
app.post('/buscar', (req, res) => {
```

```
  try {
```

```
    const { array, elemento } = req.body;
```

```
    if (!Array.isArray(array)) {
```

```
      return res.status(400).json({
        estado: 400,
        mensaje: "Debe enviar un array"
      });
```

```
    }
```

```
    const indice = array.indexOf(elemento);
```

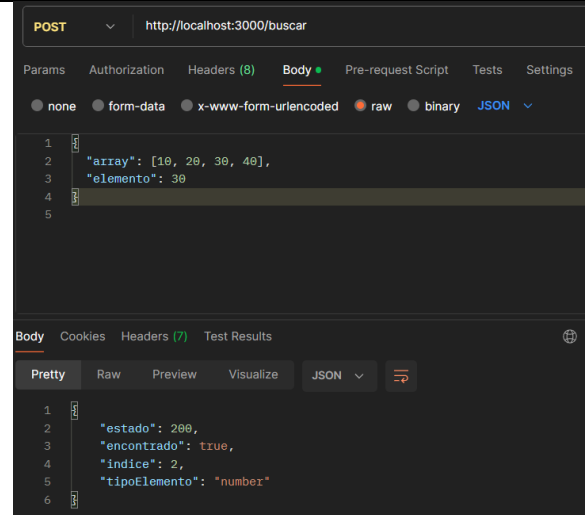
```
    res.status(200).json({
      estado: 200,
      encontrado: indice !== -1,
      indice,
      tipoElemento: typeof elemento
    });
```

```
  } catch (error) {
```

```
    res.status(500).json({
      estado: 500,
      mensaje: "Error interno"
    });
```

```
  }
```

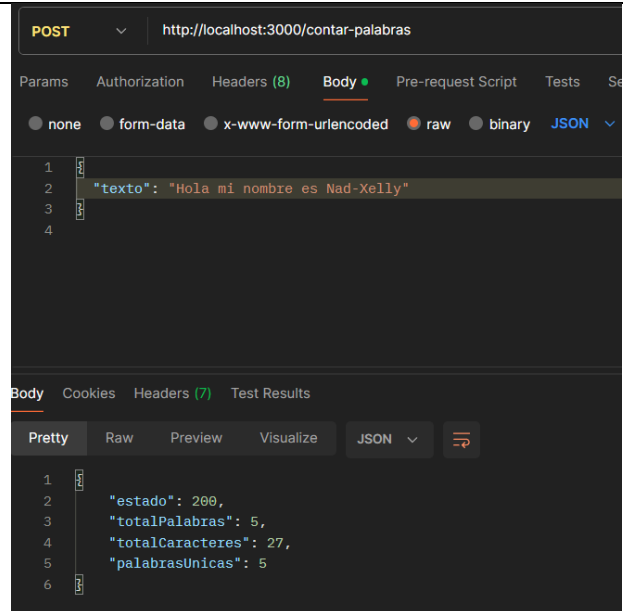
```
});
```



Ejercicio 7: Contador de Palabras

// Endpoint POST /contar-palabras que reciba { "texto": "string" } // Responder { "estado": valor, "totalPalabras": number, "totalCaracteres": number, "palabrasUnicas": number }

```
app.post('/contar-palabras', (req, res) => {
  try {
    const { texto } = req.body;
    if (!texto || typeof texto !== 'string') {
      return res.status(400).json({
        estado: 400,
        mensaje: "Texto inválido"
      });
    }
    const palabras = texto.trim().split(/\s+/);
    const unicas = new Set(palabras.map(p =>
      p.toLowerCase()));
    res.status(200).json({
      estado: 200,
      totalPalabras: palabras.length,
      totalCaracteres: texto.length,
      palabrasUnicas: unicas
    });
  } catch (error) {
    res.status(500).json({
      estado: 500,
      mensaje: "Error interno"
    });
  }
});
```



```
app.listen(PORT, () => {
  console.log(` Servidor ejecutándose en http://localhost:${PORT} `);
});
```